EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh

Mini Project Report

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology
in
Computer Engineering

by

Miti Shah	**UID 2024300230**
Twissha Shah	**UID 2024300231**
Tanvi Sheth	**UID 2024300239**

T.E. Semester V • Mini Project – I • Group 20

under the guidance of

Prof. Anuj Tawari



Department of Computer Engineering

Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (West), Mumbai – 400058

Academic Year 2026–27

Approval Certificate

This is to certify that the Mini Project entitled **"EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh"** by **Miti Shah**, **Twissha Shah** and **Tanvi Sheth** is approved for the partial fulfillment of Mini Project – I for the Third Year Mini Project towards obtaining the Degree of Bachelor of Technology in Computer Engineering.

Project Guide

_____
(Signature)

Name: _____

Date: _____

External Examiner

_____
(Signature)

Name: _____

Date: _____

_____
Head of Department

_____
Principal

Seal of the Institute

Declaration

We wish to state that the work embodied in this report titled "EcoMesh: A Distributed Peer-to-Peer Disaster Relief Communication Mesh" forms our own contribution to the work carried out under the guidance of Prof. Anuj Tawari at the Sardar Patel Institute of Technology. We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. The report distinguishes the features that are implemented and tested from those that are proposed, and every reported result is supported by stored evidence.

Miti Shah	**Twissha Shah**	**Tanvi Sheth**
UID 2024300230	UID 2024300231	UID 2024300239

Date: _____

Place: Mumbai

Acknowledgement

We would like to express our sincere gratitude to our project guide, Prof. Anuj Tawari, for the direction, the regular reviews and the candid feedback that shaped this project from a problem statement into a working, measured system. His insistence that every claim be backed by evidence is reflected throughout this report.

We thank the Head of the Department of Computer Engineering and the Principal of the Sardar Patel Institute of Technology for providing the environment, the laboratory facilities and the opportunity to carry out this mini project. We are grateful to the faculty who coordinated the evaluation phases and the report guidelines, and to the examiners whose comments at the earlier phases improved the design.

Finally, we thank our classmates, who tested the demonstration and gave honest feedback on the interface, and our families for their support throughout the semester.

Miti Shah
Twissha Shah
Tanvi Sheth

Abstract

Problem. When an earthquake, fire or building collapse destroys communication infrastructure, every mainstream messaging application fails with it, even when the devices of the trapped people can still reach each other. Connectivity after a disaster survives only in scattered pockets and normally helps only the person who holds it.

Motivation and objectives. Rescue is time-critical, so the aim is a system that makes one person's connectivity serve everyone and lets an SOS overtake routine traffic at every hop: self-organising, link-quality-weighted routing, reliable delivery with a delay-tolerant fallback, authenticated and encrypted messages, and measured performance.

Proposed solution. EcoMesh makes every participating device an endpoint and a relay. A message hops from device to device until it reaches a gateway with an outside link, a rescue device, or a store-and-forward queue that holds it until a path appears. No internet, tower or server is needed.

Technologies and engineering concepts. Python with `asyncio` (one event-loop thread per node, no locks); a custom 52-byte binary protocol; link-state gossip; Dijkstra routing over the cost $1 + 5(1 - q) + 3f$; a per-hop priority queue; ACK with retry and route recomputation; AES-256-GCM and Ed25519; per-node SQLite logs; FastAPI and WebSocket; a React web application; and a transport interface so one engine runs on an in-memory simulator, on loopback sockets and on separate laptops.

Key implementation features. Neighbour discovery with measured link quality; network-wide link-state flooding with sequence numbers; duplicate suppression; bounded store-and-forward that never evicts an SOS for routine traffic; signatures that survive multi-hop relaying; a survivor screen with a hold-to-send SOS button and a rescue console.

Testing methodology. 312 automated tests (unit, integration, boundary, fuzz, replay, concurrency and a three-node mesh over real TCP), a requirement-to-test table for 68 requirements, seven simulator scenarios with seeded repeated runs, loss and scale sweeps up to 65 nodes, and measurements on five real processes over real sockets.

Results. On a 40-node mesh with 20 % per-link loss and injected failures, 92.6 % of messages were delivered (50 runs). Weighted routing delivered 99.2 % against 57.4 % for hop-count routing on identical seeds. Under load an SOS had a 4.4× lower median latency than routine traffic. On five live processes an SOS arrived in a median of 47 ms, the sender re-routed around a failed relay in a median of 4.0 s, and every held message was delivered after a gateway outage. Convergence and recovery met their targets on average but not in every run, and delivery fell below 90 % above about 43 nodes at 20 % loss.

Practical applicability and limitations. The system is directly usable as an institutional safety network on laptops. The radio link is emulated by a neighbour-restricted LAN, so it does not yet run phone to phone; a Bluetooth or Wi-Fi Direct transport, adaptive timers, a timed usability study and per-pair key agreement are the main future work.

Table of Contents

Abstract v

Table of Contents vi

List of Figures vii

List of Tables vii

List of Abbreviations and Symbols viii

1 Introduction and Problem Definition 1

2 Literature Review and Study of Existing Systems 3

3 Software Requirements Specification 5

4 System Architecture and Design 7

5 Algorithm Design and Data Structures 13

6 Subject-Wise Engineering Concepts and APNI Leap Alignment 16

7 Implementation and Module-Wise Coding 18

8 Scalability, Performance and Resource Management 21

9 Testing and Validation 24

10 Results, Demonstration and Discussion 27

11 Conclusion and Future Scope 30

References 31

Appendix: Research Paper / Poster and Plagiarism Report 32

List of Figures

Figure 1 Block diagram: one EcoMesh node and its three interchangeable transports . . . 7
Figure 2 Level-0 (context) data flow diagram . . . 8
Figure 3 Level-1 data flow diagram of one node . . . 9
Figure 4 Use case diagram . . . 9
Figure 5 Class diagram: `Node` as the composition root . . . 10
Figure 6 Activity diagram: the packet receive path . . . 10
Figure 7 Sequence diagram: an SOS relayed hop by hop . . . 10
Figure 8 State machine of the message lifecycle and deployment diagram . . . 11
Figure 9 Entity–relationship diagram of the per-node database . . . 11
Figure 10 Flowchart: originate, acknowledge, retry and retain a message . . . 14
Figure 11 Flowchart: next-hop selection, including `RESCUE__` . . . 14
Figure 12 Delivery, routing comparison, scale and recovery results . . . 22
Figure 13 The survivor's screen: delivered, held and flushed . . . 28
Figure 14 The rescue console, before and during a gateway outage . . . 28
Figure 15 The engine web application: rescue dashboard and routing page . . . 29

List of Tables

Table 1 Key performance indicators . . . 2
Table 2 Twelve studies: technique, contribution and relevance to EcoMesh . . . 3
Table 3 Existing systems and routing protocols compared with EcoMesh . . . 4
Table 4 Functional requirements by feature . . . 6
Table 5 Modules, purpose (input → output) and dependencies . . . 8
Table 6 The 52-byte packet header . . . 8
Table 7 Sample rows of `messages` for one SOS from A to C in a real run: each node holds its own view of the same message id . . . 12
Table 8 Threat model and countermeasures . . . 12
Table 9 Data structures and why they were chosen . . . 15
Table 10 Time complexity of the critical operations . . . 15
Table 11 Subjects, concepts applied, modules and evidence . . . 17
Table 12 Modules: code reference, logic, dependencies and tests . . . 19
Table 13 Commits per team member, read from `git log` over all remote branches (53 commits) . . . 20
Table 14 Results of the seven simulator scenarios . . . 22
Table 15 Scalability, 10 to 65 simulated nodes . . . 23
Table 16 Live devices: 5 operating-system processes, real TCP sockets, real timers, on one laptop . . . 23
Table 17 Representative test cases . . . 25
Table 18 Significant defects, causes, corrective actions and retesting . . . 26
Table 19 Validation of the key performance indicators of Table 1 against measured data . . . 26
Table 20 Feature status . . . 27
Table 21 Objectives, supporting evidence and status . . . 29

List of Abbreviations and Symbols

Term	Meaning
ACK	Acknowledgement packet returned by the destination along the reverse path
AES-256-GCM	Advanced Encryption Standard, 256-bit key, Galois/Counter Mode (authenticated encryption)
AODV	Ad hoc On-Demand Distance Vector routing (RFC 3561)
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface (used by FastAPI/uvicorn)
BLE	Bluetooth Low Energy
DAA	Design and Analysis of Algorithms
DBMS	Database Management System
DFD	Data Flow Diagram
DTN	Delay-Tolerant Networking (RFC 4838)
Ed25519	Edwards-curve Digital Signature Algorithm over Curve25519
ER	Entity–Relationship (diagram)
ESE	End Semester Evaluation
FR / NFR	Functional / Non-Functional Requirement
GATEWAY	Node role: has a link to the outside world; valid terminal for `RESCUE__`
HELLO	Periodic one-hop beacon used for neighbour discovery
KPI	Key Performance Indicator
LAN	Local Area Network
MANET	Mobile Ad hoc Network
OLSR	Optimized Link State Routing (RFC 3626)
OS	Operating System
PWA	Progressive Web App
RESCUE	Node role: rescue-team device; valid terminal for `RESCUE__`
`RESCUE__`	Symbolic destination meaning “the nearest GATEWAY or RESCUE node”
SF / S&F	Store-and-Forward
SOS	Highest message priority (value 0)
SRS	Software Requirements Specification
TCP / UDP	Transmission Control Protocol / User Datagram Protocol
TTL	Time To Live (remaining hop allowance of a packet)
UML	Unified Modeling Language
WAL	Write-Ahead Logging (SQLite journal mode)
V, E	Number of vertices (nodes) and edges (links) of the mesh graph
q	Link quality in $[0, 1]$: fraction of the last 20 expected HELLOs received
f	Recent failures: ACK timeouts attributed to a link in the last 60 s
C_{max}, T_{hold}	Store-and-forward bounds: maximum count (500) and maximum age (1800 s)

CHAPTER 1

Introduction and Problem Definition

1.1 Project Background

After an earthquake, fire or building collapse, cellular towers and internet links fail or are overwhelmed, yet connectivity rarely drops to zero: it survives in pockets (one person near a window with a bar of signal, a UPS-powered access point, a rescue team's own equipment) and normally helps only the person holding it. **EcoMesh** makes every participating device an endpoint *and* a relay, so a message hops from device to device until it reaches its destination, a *gateway* with an outside link, or a *rescue* device, with no internet and no server.

1.2 Problem Statement

WhatsApp, Signal, Telegram and SMS all need a message to reach a server or tower elsewhere; a Wi-Fi router with a cut uplink breaks every one of them although the devices on it can still reach each other. People trapped in a building therefore cannot signal their condition, share the one weak signal, coordinate, or be located efficiently. Existing offline systems flood or use hop-count routing, offer no message priority, publish little measurement, and in one widely used case lacked effective confidentiality and authenticity [1] (Chapter 2).

1.3 Motivation

Rescue is time-critical: survival depends on how fast rescuers learn that someone is there, where and in what condition. A system that makes one person's connectivity serve everyone and lets an SOS overtake routine traffic at every hop shortens that time. Beneficiaries are trapped occupants, rescue teams, and institutions (hospitals, campuses, hostels, factories, stadiums) that could pre-deploy it.

1.4 Objectives

O1 *Self-organising mesh*: neighbours found automatically, every node learns the whole graph (convergence $\leq$ 15 s). O2 *Weighted routing* that beats hop count on identical scenarios. O3 *Priority*: SOS overtakes routine traffic at every hop ($\geq 3\times$ lower median latency under load). O4 *Reliability*: $\geq 90\,\%$ delivery on 40 nodes at 20 % loss and recovery within 10 s of a relay failure. O5 *Store-and-forward*: no route means retained, never dropped (100 % flushed). O6 *Security*: confidential, tamper-evident payloads; forged SOS rejected. O7 *Evidence*: every metric is a query over per-node logs. O8 *Integration*: web front end, REST/WebSocket back end and engine running on several devices.

1.5 Scope of the Project

Included: the in-scope features of the project plan: discovery, a binary wire protocol, link-state gossip, Dijkstra over a link-quality cost, per-hop priority, TTL, dedup, ACK/retry and heartbeat, node roles, bounded store-and-forward, AES-256-GCM and Ed25519, per-node SQLite logs, `asyncio` concurrency, a simulator with metrics, and survivor and rescue interfaces. **Excluded:** a real Bluetooth LE / Wi-Fi Direct transport, NAT traversal, voice, video and files, more than about 50 nodes, machine learning and a native mobile app. **Users:** trapped user, relay user, gateway operator, rescue coordinator. **Environment and scale:** laptops running Python 3.10 on a shared network; 5 live devices and up to 65 simulated nodes. The following statement is repeated verbatim because it is a design decision, not an omission:

> *The mesh currently runs over LAN sockets, which stand in for Bluetooth Low Energy or Wi-Fi Direct. On LAN we restrict each node's visible neighbour set to emulate limited radio range. This lets us develop and measure the routing, prioritisation and reliability logic without radio hardware. Discovery, routing, reliability and encryption are all transport-agnostic — replacing LAN sockets with a radio transport does not change any layer above it.*

1.6 Proposed Solution

Each device runs one node (a Python process whose `asyncio` loop runs discovery, link-state gossip, receive, send, store-and-forward and an API server). A message leaves through one of three exits: a **gateway** with an outside link (reaching any gateway counts as arrival); a **rescue** device that comes into range (every pending SOS flows in); or **store-and-forward**, where a node with no route keeps the message until one appears, so a person walking out of the building physically carries others' messages. Paths are chosen by Dijkstra over a graph whose edge weights penalise weak and failing links (Chapter 5).

1.7 Novelty and Originality

A **priority-aware, link-quality-weighted mesh with a delay-tolerant fallback, evaluated empirically**: SOS priority re-applied at every hop; cost derived from measured HELLO reception and from ACK failures, applied immediately; one transport interface so the same engine runs on a simulator, on loopback and on separate laptops; and every number is a query over per-node logs, with weighted and hop-count routing compared on identical seeds.

1.8 Feasibility Analysis

Technical: mature open-source components; the main risk (debugging routing over live sockets) was removed by building the simulator first. **Economic:** existing laptops, open-source software, no server, so there is no recurring cost. **Operational:** one command starts a node; the survivor screen has one prominent control. **Time/resources:** three members, one semester, about 336 person-hours, delivered in four tagged phases.

1.9 Social Relevance and Practical Applications

The realistic adoption path is *institutional pre-deployment*: hospitals, campuses, hostels, factories and stadiums install it on staff and resident devices as safety compliance, and rescue teams carry it as standard kit, so the rescue exit works even if no civilian has it. The benefit is faster location of survivors at no infrastructure cost.

1.10 Success Criteria and Key Performance Indicators

The targets of Table 1 are the quantified requirements of the SRS, fixed before testing; Section 9.10 validates them against measured data.

Table 1: Key performance indicators

KPI	Definition	Target	Measured by
K1 Delivery	Delivered ÷ sent, 40 nodes, 20 % loss	$\geq 90\,\%$	seeded simulator runs
K2 Convergence	Every node's graph holds every node	$\leq 15\,\mathrm{s}$	simulator, live
K3 Recovery	Relay death until a route avoiding it	$\leq 10\,\mathrm{s}$	50 trials, live
K4 SOS advantage	Median routine ÷ median SOS latency under load	$\geq 3\times$	burst scenario
K5 Store-and-forward	Retained messages flushed on return	100 %	simulator, live
K6 Weighted vs hop count	Delivery on identical seeds	weighted better	paired runs
K7 Scale	Nodes within the targets	≥ 50	sweep 10–65 nodes
K8 UI freshness	Event to screen	$\leq 2\,\mathrm{s}$	API time, poll interval
K9 Correctness	Tests passed	100 %	repeated runs
K10 Security	No relay decryption; forged SOS and tampering rejected	all hold	automated tests

CHAPTER 2

Literature Review and Study of Existing Systems

2.1 Study of Existing Systems

Offline messengers. *Bridgefy*, a Bluetooth mesh messenger used in large protests, is analysed as Study 11 in Table 2; *FireChat* gave phone-to-phone messaging and is discontinued; *Briar* has no server and synchronises offline over Bluetooth or Wi-Fi with end-to-end encryption [16]. **Dedicated radios.** *goTenna Mesh* uses paired radios; *Meshtastic* runs on LoRa with managed flood routing, suppressing a rebroadcast when a neighbour already repeats it [17]. **Research and humanitarian systems.** The *Serval Project* builds disaster phone meshes with *Rhizome* store-and-forward [18]; the *DTN architecture* provides store-carry-forward across intermittent links [10, 11]. "n/d" means not documented in the sources consulted.

2.2 Review of Research Papers and Technical Resources

Table 2 records the twelve sources studied: technique, contribution and relevance.

Table 2: Twelve studies: technique, contribution and relevance to EcoMesh

#	Source	Technique and contribution	Relevance and limitation
1	DSDV, Perkins and Bhagwat 1994 [2]	Proactive distance vector; sequence numbers prevent loops	Routes ready before the first SOS; hop-count metric, overhead grows with churn
2	DSR, RFC 4728 [3]	On-demand source routing; the full route travels in every packet	Carried path gives loop detection and "relayed via n"; no retention
3	AODV, RFC 3561 [4]	On-demand distance vector; flooded request, reply on the reverse path	Hop-count metric; no retention, no priority
4	OLSR, RFC 3626 [5]	Proactive link state; multipoint relays cut flooding	Closest family to EcoMesh; hop count by default
5	OSPF v2, RFC 2328 [6]	Sequence-numbered link-state flooding; Dijkstra on every router	EcoMesh reuses the structure with a measured cost; built for wired networks
6	Babel, RFC 8966 [7]	Loop-free, metric-agnostic distance vector for wireless meshes	The wireless metric must be pluggable; routing only
7	ETX, De Couto et al. 2003 [8]	Path metric from measured delivery ratios; minimum-hop paths use lossy links	Strongest basis for our cost; we add an ACK-failure penalty
8	Epidemic routing, Vahdat and Becker 2000 [9]	Replicate a message to every node met	High delivery, high buffer use; we keep a bounded queue
9	DTN architecture, Fall 2003; RFC 4838 [10, 11]	Store-carry-forward with custody transfer	Basis of our third exit; no priority or link quality
10	Disaster-response networks, Reina et al. 2015 [12]	Survey of MANET, VANET, DTN and sensor networks for disasters	Confirms the framing and the need for evaluated systems
11	Breaking Bridgefy, Albrecht et al. 2021 [1]	Practical attacks on a deployed mesh messenger: tracking, no authenticity, shutdown	Sets the threat model; justifies encryption and signatures
12	AES-GCM, Ed25519, Dijkstra [13–15]	AES-GCM; fast high-security signatures; shortest paths	The primitives and routing kernel used

2.3 Comparative Analysis

Table 3 compares the systems of Section 2.1 and the routing protocols of Table 2 with EcoMesh on six criteria: the infrastructure each needs, how it routes, whether it prioritises messages, whether it retains a message that has no route, how it protects content, and (from Section 2.1) whether it is available today.

Table 3: Existing offline-communication systems and routing protocols compared with EcoMesh (n/d = not documented in the sources consulted)

System	Infrastructure	Routing	Priority	Store-fwd	Encryption
Bridgefy	Bluetooth LE phones	n/d	n/d	n/d	none effective in analysed version [1]
Briar	none (Bluetooth/Wi-Fi, Tor)	direct device sync	n/d	on contact	end-to-end [16]
Meshtastic	LoRa radios	managed flood [17]	n/d	n/d	channel-level
Serval	Wi-Fi ad hoc phones	B.A.T.M.A.N., later own	n/d	yes (Rhizome) [18]	n/d
DTN	none	architecture only	bundle class of service	yes (custody)	separate specification
AODV, DSR, DSDV, OLSR, OSPF, Babel	none	hop count, configured or pluggable metric	none	none or short	not in protocol
EcoMesh	**none**	**link state, weighted Dijkstra**	**3 levels, every hop**	**yes, bounded**	**AES-GCM + Ed25519**

2.4 Limitations of Existing Systems

Hop count, the default metric of the surveyed protocols, picks lossy links [8]; none prioritises an emergency message at every hop; ad hoc routing drops a message with no route while delay-tolerant systems ignore link quality and priority; and deployed messengers publish little measurement, one with no effective confidentiality or authenticity [1].

2.5 Identified Research or Engineering Gap

No surveyed system combines priority-aware, link-quality-weighted routing with a delay-tolerant fallback and is evaluated quantitatively under controlled failure.

2.6 Proposed Improvements

EcoMesh routes over the cost $1+5(1-q)+3f$ (Section 5.2), re-applies SOS priority at every hop, keeps unroutable messages in a bounded queue that never evicts an SOS for routine traffic, authenticates and encrypts with vetted primitives, and measures all of it, including a paired weighted-versus-hop-count experiment.

CHAPTER 3

Software Requirements Specification

3.1 Purpose and Scope of the SRS

This chapter condenses the IEEE 830 SRS of Phase II (`docs/SRS_EcoMesh_Group20.pdf`): *what* EcoMesh must do, not how. The scope is one deployment of cooperating nodes with ten features (discovery, message composition, forwarding, routing, priority, reliability, store-and-forward, security, logging, rescue dashboard) and no server in any mode.

3.2 Intended Users and Stakeholders

The *trapped user* originates an SOS and needs one prominent control and no configuration; the *relay user* carries others' traffic and must not be able to read it; the *gateway operator* holds the outside link; the *rescue coordinator* monitors the SOS queue, topology and statistics; the *administrator* assigns node identifiers and who hears whom.

3.3 Overall Product Description

Every node runs the same program: HELLO beacons build a neighbour table, link-state reports a graph; Dijkstra selects next hops; a per-hop priority queue orders transmissions; ACK, retry, dedup and store-and-forward give reliability; AES-256-GCM and Ed25519 give confidentiality and authenticity; an embedded web server exposes the node to a browser. A configured peer list replaces multicast where needed (DIS-7).

3.4 Functional Requirements

The 68 functional requirements fall into the ten features of the SRS. Each requirement has a unique identifier (DIS-1, DIS-2, ...) and an acceptance criterion: the automated tests that must pass and, for screens, a manual check. Table 4 gives each feature's input, processing and output and how its requirements are verified; the Phase II SRS (`docs/SRS_EcoMesh_Group20.pdf`) lists all 68 individually, and the tests that accept each one are named in `docs/report/evidence/fr_table.py`.

3.5 Non-Functional Requirements

Performance: at least 90 % delivery on 40 nodes at 20 % loss; convergence within 15 s; delivery resumed within 10 s of a node failure; an SOS measurably faster under load. **Scalability:** at least 50 nodes without reconfiguration. **Availability and reliability:** no single point of failure; no message discarded while retention capacity remains. **Security and privacy:** payloads unreadable by relays; tampering and forged SOS rejected; every field validated; private keys never transmitted. **Usability:** one prominent SOS control needing a sustained press; status visible within 2 s. **Maintainability and portability:** one module per concern; the engine is independent of the link technology. **Resources:** one process on one event-loop thread with bounded queues (Chapter 8).

3.6 Hardware and Software Requirements

Hardware: laptops (Windows 11) on a Wi-Fi network or a phone hotspot; no server. **Software:** Python 3.10 with `asyncio`, `cryptography` (AES-256-GCM, Ed25519) and SQLite in WAL mode; FastAPI, uvicorn and `websockets`; React 18 with Vite, Tailwind and d3-force, installable as a PWA; pytest and pytest-asyncio; Git and GitHub.

Table 4: Functional requirements by feature: input, processing, output and verification (Auto = automated tests; UI = automated tests plus a manual check of the screen; Insp. = code inspection; Man. = manual check; Part. = partly implemented)

Feature (IDs)	**Input → processing → output**	**Verification**
Peer discovery (DIS-1–7)	HELLO beacons and silences → update the neighbour table, estimate link quality over a 20-HELLO window, prune after the timeout → peer-up/peer-down events and link quality for routing	7 Auto
Message composition (MSG-1–7)	text and destination, or a sustained press of the SOS control → build a packet with a unique id and timestamp, encrypt, sign, queue → DATA packet sent; delivery state shown	4 Auto, 3 UI
Multi-hop forwarding (FWD-1–7)	DATA or ACK from a neighbour → validate, drop duplicates, check hop allowance and loop, append own id, queue by priority → forwarded, or dropped with a logged reason	7 Auto
Adaptive routing (RTE-1–8)	link-state reports and a destination → merge into the graph, Dijkstra over the weighted cost, resolve `RESCUE__` → least-cost next hop, recomputed on topology change	8 Auto
Priority handling (PRI-1–6)	packets of different priority → priority queue by priority then age, at every hop → SOS sent before routine traffic	4 Auto, 1 UI, 1 Man.
Reliability (REL-1–8)	delivered or unacknowledged packets, duplicates, missing beacons → ACK, timer, retry on a recomputed route, dedup, failure detection → bounded retries; duplicates dropped	8 Auto
Store-and-forward (SNF-1–6)	a message with no route → bounded retention by count and age, periodic flush, lowest priority evicted first → delivered when a path appears, or expired and logged	4 Auto, 1 Man., 1 Part.
Message security (SEC-1–7)	payload and identity fields → AES-256-GCM, Ed25519 signature, verification and decryption on receipt → tampered or forged packets rejected	6 Auto, 1 Insp.
Logging and history (LOG-1–6)	every message and network event → write to the node's SQLite database → history and delivery statistics by query	4 Auto, 1 Insp., 1 Part.
Rescue dashboard (DSH-1–6)	node state and events → topology, SOS queue and statistics from the graph and logs → live graph; SOS queue by priority then age	6 UI
Total (68)		**52 Auto, 10 UI, 2 Insp., 2 Man., 2 Part.**

3.7 Constraints and Assumptions

Constraints: one semester (17 August to 2 November 2026); three members; no radio hardware, so a neighbour-restricted LAN emulates the radio; about 50 nodes; no machine learning or native app. **Assumptions:** the software is installed before an incident; every device can reach another, directly or through others; batteries last the incident; a gateway or rescue node exists for messages to leave; node identifiers are unique.

3.8 Use Cases and User Stories

UC1: a trapped user holds the SOS control; the highest-priority message is sent and its status shown. **UC2:** with no route the message is retained and flushed when a path appears. **UC3:** a relay user forwards automatically without reading the payload. **UC4:** a gateway node advertises its role. **UC5:** a rescue coordinator sees the SOS queue with routes, and the topology. **UC6:** an administrator sets who hears whom in a topology file.

CHAPTER 4

System Architecture and Design

4.1 Overall System Architecture

A node is one operating-system process. Inside it `asyncio` runs discovery (HELLO every 3 s, timeout 10 s), link-state gossip (every 5 s), the receive path, the sender (priority queue), the store-and-forward flush (every 10 s) and the API server on one event-loop thread. Every activity reaches the network only through the `Transport` interface (Figure 1). Shared mutable state (neighbour table, mesh graph, pending-ACK map) lives on one thread, so there are no locks and no data races by construction.

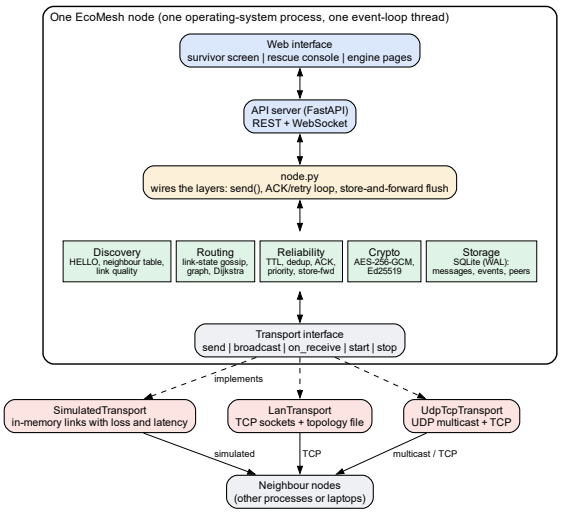


Figure 1: Block diagram: one EcoMesh node and its three interchangeable transports

4.2 Component and Module Description

Table 5 gives each module's purpose, input and output, and dependencies. **Wire protocol.** A fixed big-endian header (Table 6) is followed by the traversed path, the payload and an optional 64-byte signature. The path is carried on the wire for loop detection and so that the interface can show "relayed via n nodes". HELLO is a one-hop shout, LINK_STATE is unicast to neighbours, and DATA and ACK use TCP per hop, so each hop is ordered and reliable though the end-to-end path is not.

4.3 Architecture Selection and Justification

Layered and transport-agnostic. One boundary separates "how bytes move" from everything else. It gives a deterministic simulator, socket-free unit tests and an honest answer to the radio objection (a BLE transport is a fourth implementation, not a rewrite). **Single-threaded `asyncio` rather than threads.** Four activities share three structures; on one loop there are no locks or races, so correctness does not rest on lock discipline. **Peer-to-peer rather than client–server.** A server is a single point of failure and the very infrastructure being argued against. **Link state rather than flooding or on-demand discovery.** Flooding collapses under load and on-demand discovery delays the first emergency packet. **Per-node SQLite (WAL).** File-based, no server, and the API can read while the engine writes.

Table 5: Modules, purpose (input → output) and dependencies

Module	Purpose: input → output	Depends on
`packet.py`	pack, parse and validate the 52-byte header, path, payload and signature: bytes ↔ `Packet`	–
`transport*.py`, `lan_transport.py`	the transport boundary and its simulated, multicast/TCP and multi-device implementations: bytes between neighbours	`packet`
`discovery.py`	HELLOs → neighbour table, peer-up and peer-down events, link quality	`transport`
`relay.py`	received packets → deliver, forward in priority order, or drop with a reason	`reliability`
`routing.py`	link-state reports → mesh graph, least-cost next hop, `RESCUE__` terminal	`discovery`
`reliability.py`	packets → dedup decision, priority order, ACK timers, retained messages	`packet`
`crypto.py`	payload and identity bytes → ciphertext, signature, verification result	`cryptography`
`store.py`	messages, events and peers → SQLite rows and query results	`sqlite3`
`node.py`	wires every layer; `send()`, ACK/retry loop, flush loop	all above
`sim/`, `api/`	scenario scripts → metrics; REST/WebSocket server	`node`
`ui/`, `story/`	React application; one-device program with survivor and rescue screens	`api`, `node`

Table 6: The 52-byte packet header

Off.	Size	Field	Meaning	Off.	Size	Field	Meaning
0	2	magic	`0x4D45` “ME”	24	8	src	node id, ASCII
2	1	version	1	32	8	dst	node id / `RESCUE__`
3	1	type	HELLO, LINK_STATE, DATA, ACK	40	8	timestamp	Unix ms
4	1	priority	0 SOS, 1 NORMAL, 2 STATUS	48	2	payload_len	bytes
5	1	ttl	per-hop decrement (10)	50	1	path_len	ids in path
6	1	flags	enc., needs_ack, s-f, signed	51	1	attempt	retry counter
7	1	hop_count	per-hop increment	8	16	msg_id	UUIDv4

4.4 Data Flow Diagram (DFD)

Figure 2 is the context-level DFD: the system boundary contains one node and no server. Figure 3 is the level-1 decomposition into six processes (discover, build graph, receive, forward, originate/retry/retain, log and serve) and five data stores.

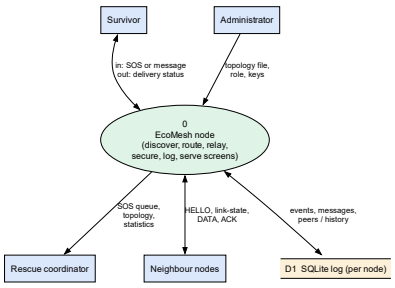


Figure 2: Level-0 (context) data flow diagram

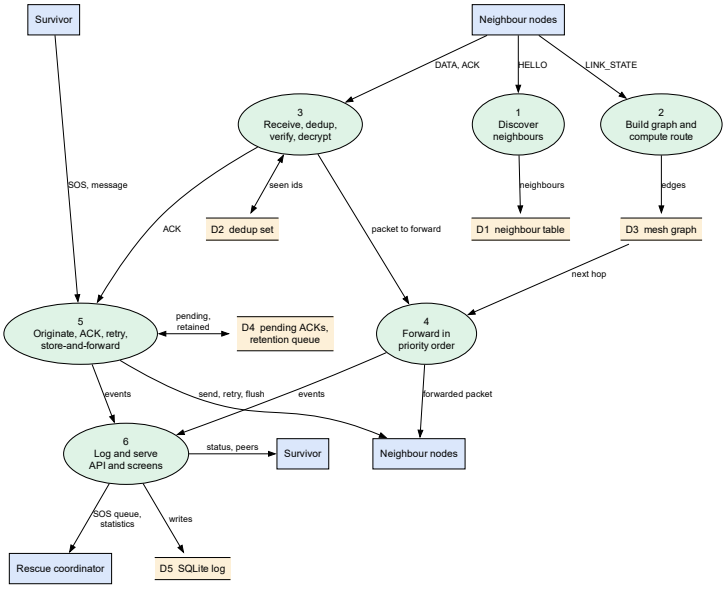


Figure 3: Level-1 data flow diagram of one node

4.5 UML Diagrams

The six UML diagrams of Phase II are reproduced below as they match the implemented design. **Use case diagram** (Figure 4). The actors are the trapped user, the passive relay user, the gateway node, the rescue coordinator and the administrator. Raising an SOS includes sending a message; holding and flushing undelivered messages extends sending when no route exists. **Class diagram** (Figure 5). `Node` owns the discovery, gossip,

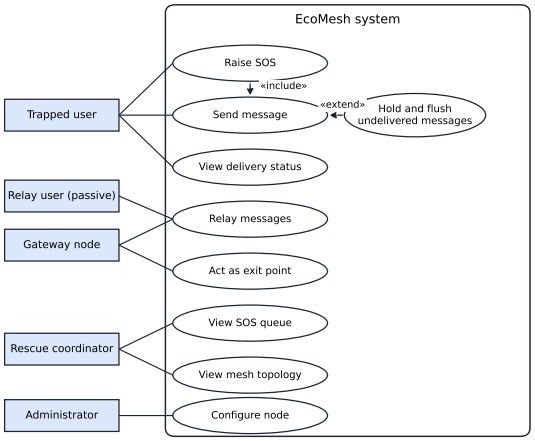


Figure 4: Use case diagram

routing, relay, ACK-tracking, retention and storage components (filled diamonds); the router and gossip share one `MeshGraph`; the relay owns its priority queue and uses a dedup set; three classes implement the `Transport` interface. **Activity diagram** (Figure 6). A packet is validated and checked for duplicates; if it is for this node its signature is verified, its payload decrypted, and it is logged, acknowledged and delivered; otherwise the hop allowance, loop and route checks decide between dropping it and re-queuing it by priority. **Sequence diagram** (Figure 7). An SOS is created, encrypted and signed at L1, forwarded through L2 and the gateway (each relay applies dedup, decrements the hop allowance and appends its own id) and delivered at L5, whose

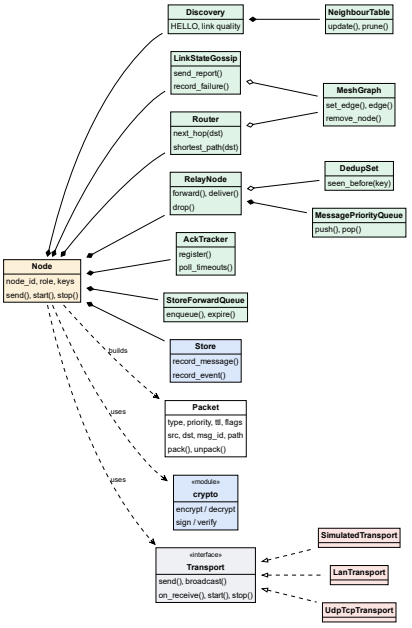


Figure 5: Class diagram: `Node` as the composition root

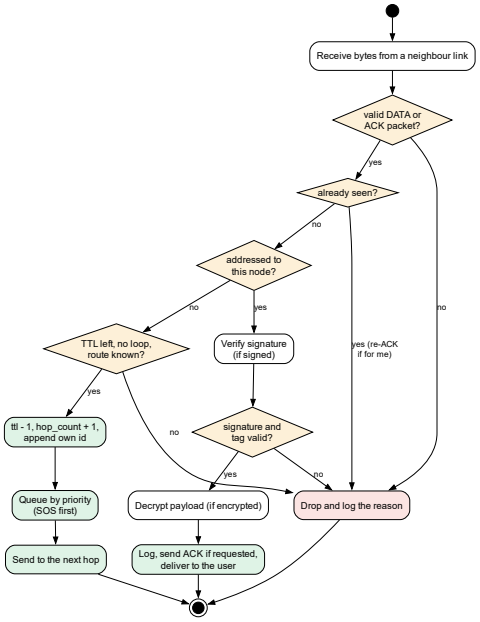


Figure 6: Activity diagram: the packet receive path

ACK returns along the reverse path. **State machine** (Figure 8, left). A message is created, then pending

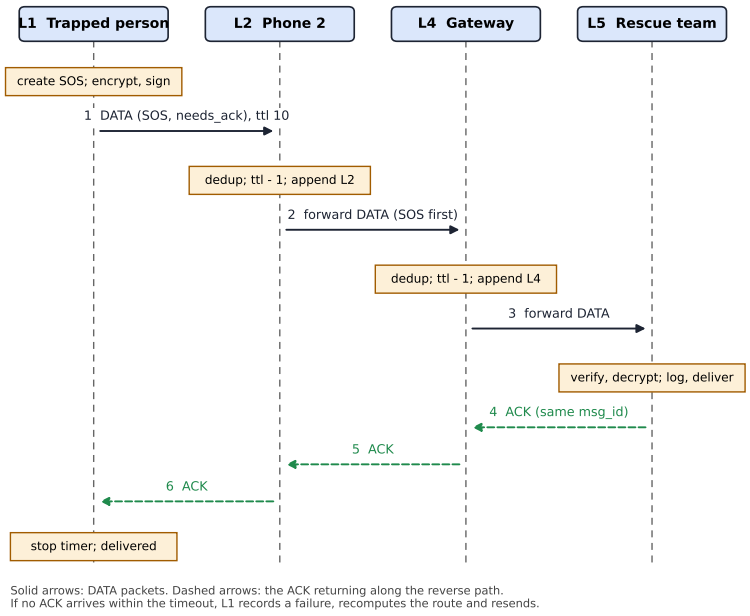


Figure 7: Sequence diagram: an SOS relayed hop by hop with the ACK returning along the reverse path

(sent, awaiting ACK) or held (no route), and ends delivered or failed; the states are the `status` values stored in the database. **Deployment diagram** (Figure 8, right). Five node processes on two laptops, each with its own engine, web screen and SQLite file, joined only by TCP links where the topology file lists them; there is no server (Section 7.7).

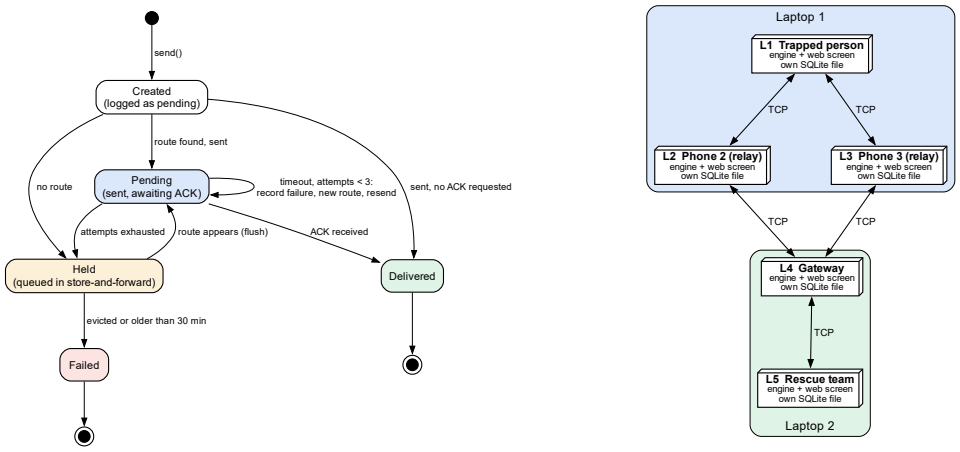


Figure 8: State machine of the message lifecycle (left) and deployment diagram of the live configuration (right)

4.6 Database Design

Each node keeps its own SQLite database (WAL mode, plain `sqlite3` and raw SQL: three tables, so an ORM would add indirection for nothing). Every metric is a query over it (SRS 6.1). **Relational schema.**

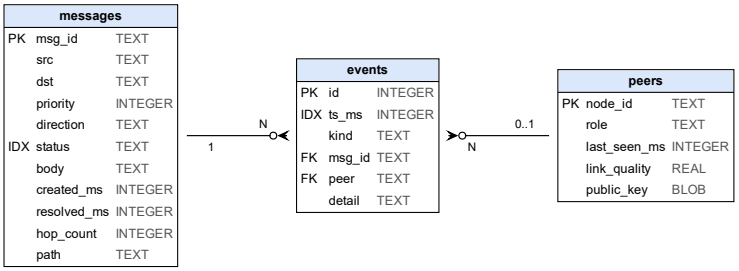


Figure 9: Entity–relationship diagram of the per-node database (PK primary key, IDX secondary index, FK logical foreign key)

`messages`(`msg_id` PK, `src`, `dst`, `priority`, `direction` in sent/received/forwarded, `status` in pending/delivered/failed/queued_sf, `body`, `created_ms`, `resolved_ms`, `hop_count`, `path`); `events`(`id` PK, `ts_ms`, `kind`, `msg_id`, `peer`, `detail`); `peers`(`node_id` PK, `role`, `last_seen_ms`, `link_quality`, `public_key`). **Keys and integrity constraints.** `msg_id` is a UUID and is the join key between a node's messages and events; `messages.body` holds plaintext only if the node is the source or destination; writes to `messages` are upserts, so a message moves pending → queued_sf → delivered without duplicate rows; `events.msg_id` and `events.peer` are logical, not declared, foreign keys because a relay logs events for messages it never stores and for peers that may have expired. **Normalisation.** Third normal form except `messages.path`, a comma-separated list kept denormalised because it is always read and written whole. **Indexing.** `idx_events_ts` serves "events since t" for the live feed; `idx_msg_status` serves the retained-message counter. **Sample records** from a real run are in Table 7; the relay's row carries no body.

Table 7: Sample rows of `messages` for one SOS from A to C in a real run: each node holds its own view of the same message id

DB	msg_id	src	dst	prio	direction	status	body	hops	path
A	8e859198...	A	C	0	sent	delivered	trapped on floor 3	1	B
B	8e859198...	A	C	0	forwarded	delivered	NULL	1	B
C	8e859198...	A	C	0	received	delivered	trapped on floor 3	1	B

4.7 Security and Error Handling Design

Table 8 maps each threat to the mechanism that counters it and to the requirement it satisfies; the paragraphs below describe each concern. **Authentication.** Nodes have no user accounts; a node's identity is its Ed25519

Table 8: Threat model and countermeasures

Threat	Mechanism	Requirement
Relay reads a message	AES-256-GCM payload; a relay without the session key reads only `dst` and `ttl`	SEC-1, 2, FWD-7
Relay modifies a message	the GCM tag fails at the destination; packet dropped and logged	SEC-5
Relay forges an SOS	Ed25519 signature over the identity fields; public keys in HELLO and gossip	SEC-3, 4, 6
Replay of a captured packet	dedup set keyed on message id (300 s)	REL-3
Malformed or hostile bytes	strict validation; `PacketError`; drop	FWD-6
Endless circulation	TTL per hop; own id in the path is a loop	FWD-3, 5
Hand-rolled cryptography	only `cryptography`-library primitives	SEC-7

key pair, distributed in HELLO and link-state reports, and every signed packet is verified against the sender's known key. **Authorization.** Authority follows the node role: only GATEWAY and RESCUE nodes are valid terminals for `RESCUE__`; any node may relay. The relay path never decrypts or stores content it does not own, and a relay that does not hold the session key sees only ciphertext; because one pre-shared key serves the whole mesh, a node that *does* hold it could read what it forwards, which per-pair key agreement would remove (Section 9.11). **Input validation.** Every packet field is range-checked before use; a violation raises `PacketError` and the packet is dropped, never forwarded. **Exception handling.** A bad packet is dropped and logged and never raised into the event loop; crypto failures collapse into one exception type and `verify` returns `False` instead of raising. **Logging.** Each drop has a named event kind (`drop_ttl`, `drop_dup`, `drop_loop`, `drop_signature_invalid`, `drop_decrypt_failed`, ...) in the node's database. **Protection of sensitive data.** Payloads are AES-256-GCM ciphertext with a random 12-byte nonce; plaintext is stored only on the originator and destination; private keys stay in local memory and are never transmitted. The signature covers the identity fields (version, type, priority, flags, src, dst, msg_id, timestamp, payload) and not the whole header, because TTL, hop count and path change at every relay and a signature over them could never verify beyond the first hop; the retransmission counter is excluded so a retry keeps its identity.

CHAPTER 5

Algorithm Design and Data Structures

5.1 Identification of Critical Operations

Ten operations decide correctness and performance: (1) neighbour discovery and link quality; (2) link-state dissemination; (3) least-cost routing; (4) the per-packet forwarding decision; (5) priority scheduling; (6) duplicate suppression; (7) send, acknowledge and retry; (8) bounded store-and-forward; (9) encryption and signing; (10) resolving `RESCUE__`. They are Algorithms 1–10 of the Phase II *Design of Algorithms*, shown here as implemented.

5.2 Algorithm Design

A1 Discovery and link quality.

```
every 3 s: broadcast HELLO(role, public_key)                        // ttl = 1
on HELLO from p:  new <- table.update(p);  if new: on_peer_up(p)
    q <- hellos_received / hellos_expected   in a 20-slot window    // in [0,1]
    if |q - last_reported| >= 0.1: report q                         // avoids recomputing
    on noise
every 1 s: for p silent > 10 s: remove p; on_peer_down(p)
```

A new neighbour is scored over the HELLOs *due* so far, otherwise every fresh link would look lossy.

A2 Link-state with re-flooding.

```
every 5 s: seq++; send report(origin=me, seq, [(p, q, failures, role, key) for p in
    neighbours]) to each neighbour
on report(origin, s, entries) from sender:
    if origin = me or s <= seq_seen[origin]: return                  // stale or own: stop
    seq_seen[origin] <- s;  replace origin's outgoing edges with entries;  learn roles and
    keys
    if changed: invalidate route cache
    forward report unchanged to every neighbour except origin and sender   // controlled re
    -flooding
```

Without re-flooding a node knows only its neighbours' neighbours; sequence numbers admit each report once per link, so flooding terminates.

A3 Dijkstra and the cost function.

```
cost(u->v) = 1 + 5 * (1 - q) + 3 * f        // f = ACK timeouts on the link in the last 60
    s
dist[src] <- 0; heap <- {(0, src)}
while heap: (d,u) <- pop_min; if u visited: continue; mark u
    for (v, e) in graph.neighbours(u): if d + cost(e) < dist[v]: dist[v] <- d + cost(e);
    prev[v] <- u; push
next_hop(dst) = path[1] of the cached shortest path, or none (=> store-and-forward)
```

Derivation of the coefficients. The base 1 is the per-hop penalty; it keeps every weight positive (Dijkstra's condition) and prefers fewer hops between equal links. The term $5(1-q)$ makes loss expensive: a link delivering 40 % of its HELLOs costs $1+5(0.6)=4.0$, more than two clean hops (2.0) or three (3.0), which is exactly where minimum-hop routing errs. The term $3f$ is steeper than the estimated-loss term because an ACK timeout is *observed* failure: one adds the cost of three extra clean hops, and it applies at once so the retry avoids the failing link.

A4 Packet handling.

```
on packet p from link sender s:
    if not parse_ok(p) or p.type not in {DATA, ACK}: drop
    for_me <- p.dst = me or (p.dst = RESCUE__ and my role in {GATEWAY, RESCUE})
    key <- (type, msg_id) if for_me else (type, msg_id, attempt)     // a retry crosses
    relays that saw the original
    if dedup.seen_before(key): drop(duplicate); re-ACK if for_me; return
    if for_me: verify signature; decrypt; log; ACK if needs_ack; deliver
    elif ttl <= 0: drop(ttl_expired)   elif me in path: drop(loop)   elif no next_hop: drop
    (no_route)
    else: ttl--; hop_count++; path.append(me); queue.push(p, priority)            // A5
```

The key includes the *type*, because an ACK carries the id of the DATA it acknowledges (a bug found in integration), and at relays the *attempt*, so a retransmission is forwarded; the destination ignores the attempt and sees each message once. Only the originator retains (A8).

A5 Priority scheduling. `push` inserts (priority, timestamp, counter) into a binary heap and `pop` returns the highest priority, then the oldest; the counter prevents ties from comparing packets. Applied at *every* hop, an SOS overtakes routine traffic repeatedly along the route. **A6 Dedup.** A hash map of keys to first-seen time, pruned after 300 s. **A7 Send, ACK, retry.** A timed-out packet with fewer than 3 attempts gets a failure recorded on its link (cost +3), a recomputed route and a resend; after 3 attempts it moves to store-and-forward (Figure 10). **A8 Bounded store-and-forward.** `enqueue` evicts, when over 500 messages, the one with the largest (priority value, age), i.e. the lowest priority and oldest; `expire` drops anything older than 1800 s regardless of priority; the flush (every 10 s) sends each message whose destination now has a route. **A9 Security.** Send encrypts with a random nonce and signs the identity bytes; receive verifies against the sender's known key, then decrypts; either failure drops the packet. **A10 `RESCUE__`.** One Dijkstra run, then the nearest reachable node whose role is GATEWAY or RESCUE; none reachable means store-and-forward (Figure 11).

5.3 Flowcharts

Figure 10 shows how a message is originated, acknowledged, retried and retained; Figure 11 shows how a next hop is chosen. The packet receive path is the activity diagram of Figure 6.

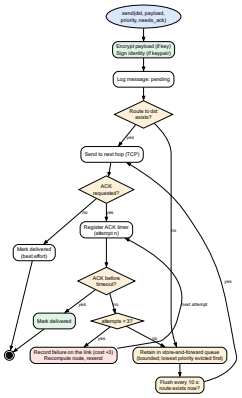


Figure 10: Flowchart: originate, acknowledge, retry and retain a message

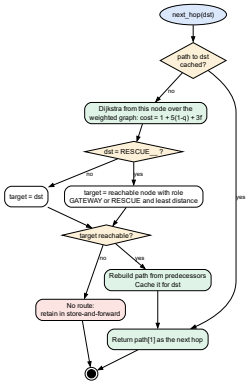


Figure 11: Flowchart: next-hop selection, including `RESCUE__`

5.4 Data Structure Selection

Table 9 lists each structure, its use and the reason.

Table 9: Data structures and why they were chosen

Structure and use	Reason
Adjacency dictionary of dictionaries: `MeshGraph`	$O(1)$ edge update and lookup, $O(\deg)$ iteration in Dijkstra; the mesh is sparse, so no matrix
Binary min-heap: Dijkstra frontier and the message priority queue	$O(\log n)$ insert and extract-min; ordering by (priority, timestamp, counter)
Hash map with timestamps: dedup set, neighbour table, route cache	$O(1)$ average membership; expiry by pruning
Fixed-capacity deque (20 slots) per neighbour	bounded sliding window for the link-quality estimate
Dictionary keyed by message id: ACK tracker, retention queue	$O(1)$ acknowledge and removal
Set: visited nodes in Dijkstra, reachable set	$O(1)$ membership

5.5 Algorithm Selection and Justification

Flooding delivers but its message count grows with the square of the node count and collapses a mesh under load. Bellman–Ford ($O(VE)$) and A* (needs a geometric heuristic) bring nothing here. Dijkstra with a binary heap is correct for non-negative weights in $O((V + E) \log V)$, and the weighted cost avoids the lossy shortcuts hop count picks (Chapter 8). A heap rather than a sorted list or FIFO gives $O(\log n)$ priority order; a hash set gives $O(1)$ duplicate checks.

5.6 Time Complexity Analysis

Table 10 gives the time complexity of the critical operations.

Table 10: Time complexity (V nodes, E links, n queued packets, q retained messages, D distinct destinations, $|N|$ neighbours)

Operation	Time
Dijkstra with the weighted cost	$O((V + E) \log V)$ in every case
Link-state merge on receiving a report	$O(\deg(v))$ plus $O(\|N\|)$ to re-flood
HELLO handling	$O(1)$ amortised
Priority enqueue / dequeue	$O(\log n)$; draining n packets $O(n \log n)$
Duplicate detection	$O(1)$ average
Loop check	$O(\|\text{path}\|)$, at most 10 here
Store-and-forward flush	$O(D\,(V + E) \log V + q)$ with the route cache; worst case $O(q\,(V + E) \log V)$
Eviction	$O(q)$ scan, only when over the bound
Packet parse and validate	$O(1)$ for the fixed 52-byte header

5.7 Space Complexity Analysis

The graph needs $O(V + E)$, the dedup set grows with traffic in its 300 s window, the retention queue holds at most 500 packets and Dijkstra needs $O(V)$ extra; only the on-disk SQLite log is unbounded.

5.8 Optimization Opportunities

Per-destination cache invalidation and incremental shortest paths; multipoint relays to cut redundant report copies [5]; short retention at relays; batched SQLite commits; and the timer changes the measurements point to (Chapter 8).

CHAPTER 6

Subject-Wise Engineering Concepts and APNI Leap Alignment

This chapter maps the Semester I–IV subjects that the project genuinely uses to the implementation; a subject the problem does not call for is marked not applicable, with the reason.

6.1 Software Engineering

Modularity: seven layers (discovery, transport, routing, reliability, crypto, storage, API) are separate modules joined only at `node.py`. *State management:* the node's mutable state is a small named set of structures, each owned by one component, and the message lifecycle is an explicit state machine (Figure 8) whose states are the stored `status` values. *Requirement traceability:* every requirement has an identifier and acceptance tests (Table 4). *Exception handling:* a bad input is dropped and logged and never raised into the event loop; cryptographic failures collapse to one exception type. *Version control:* Git with phase branches and Conventional Commits (Section 7.8).

6.2 Computer Networks

The communication architecture is peer-to-peer: each node is both host and router. The protocol is a custom binary format with a magic value, a version and a self-describing length, so a stream reader frames packets without a separate prefix (Table 6). HELLO is a one-hop broadcast, link-state reports are unicast to neighbours, and DATA and ACK use TCP per hop. `LanTransport` opens a connection per packet and prefixes it with the previous hop's identity, so flooding never echoes a report back to its sender; connect and drain are bounded by 1.5 s so a dead laptop fails fast. Errors are recovered by TCP per hop, by ACK, retry and route recomputation end to end, and by store-and-forward; failure detection uses missed HELLOs. *TCP between many clients and a central server is not applicable:* the project's requirement is the absence of a central server in every mode; the equivalent is many peers, each a server for its neighbours and a client of them, which the live deployment exercises.

6.3 Database Management Systems

Schema, primary keys, the deliberately denormalised `path` column, the two secondary indexes, upserts, logical foreign keys and WAL concurrency are in Section 4.6. *Transaction management:* every write is committed immediately, so each event is durable once logged (subject to SQLite's default synchronisation setting, which we did not tighten), and WAL lets the API read while the engine writes. *Query-driven metrics:* delivery ratio, hop histogram, per-priority latency (median and 95th percentile), retries, retained and flushed counts and the duplicate-drop ratio are queries over `messages` and `events`. *Distributed databases:* each node owns its file and there is no shared or replicated database; cross-node queries are assembled by the simulator or API reading each node's log.

6.4 Operating Systems

One node is one process; five processes ran live and communicated only over TCP and HTTP (inter-process communication). A single thread runs a cooperative `asyncio` event loop, so shared structures are touched only on that thread and there are no locks and no races by construction. Scheduling is *priority scheduling* (SOS, NORMAL, STATUS; ties oldest first) re-applied at every hop; Round-Robin was rejected because an emergency must not share a link fairly with routine traffic, and the scheduling metric is the SOS latency advantage measured in Chapter 8. I/O is non-blocking with timeouts, and memory is bounded (500 retained messages, 1800 s). On Windows with Python 3.10, `asyncio.wait_for` can swallow a task cancellation that arrives as the inner operation completes, which made `Node.stop()` hang intermittently; a cancellation-safe helper fixed it (Section 9.9).

6.5 Data Structures

Adjacency dictionaries, binary heaps, hash maps and sets, fixed-capacity deques and id-keyed dictionaries give efficient insertion, search, deletion and request handling (Table 9); the heap serves both Dijkstra's frontier and the priority queue.

6.6 Design and Analysis of Algorithms

Dijkstra's algorithm is correct here because every edge weight is at least 1, and it runs in $O((V + E) \log V)$ (Table 10). The cost function is a design contribution with derived coefficients (Section 5.2); link-state flooding terminates because sequence numbers admit each report once per link; and the weighted-versus-hop-count experiment (Chapter 8) is the empirical test of the design.

6.7 Cloud Computing

Not applicable by design: the system has no server, so there is no cloud deployment, hosted database or autoscaling, and no recurring infrastructure cost.

6.8 Mapping of Subjects to Project Implementation

Table 11 maps each subject to the concept applied, the module where it is applied and the evidence that it works.

Table 11: Subjects, concepts applied, modules and evidence

Subject	**Concept applied**	**Project module**	**Evidence**
Software Engineering	modularity, explicit state machine, error policy, traceability, version control	all; `node.py`	Table 4; Section 7.8
Computer Networks	binary protocol, framing, per-hop TCP, previous-hop identity, timeouts, failure detection	`packet.py`, `lan_transport.py`, `discovery.py`	`test_packet`, `test_lan_transport`; live test
DBMS	schema, keys, indexes, upserts, WAL, query-driven metrics	`store.py`, `api/metrics.py`	`test_store`; Section 4.6
Operating Systems	process per node, single-threaded event loop, priority scheduling, bounded memory	`node.py`, `reliability.py`	concurrency tests; priority scenario
Data Structures	heap, hash map and set, adjacency dictionary, deque	`routing.py`, `reliability.py`	`test_reliability`, `test_dijkstra`
Algorithms and Graph Theory	Dijkstra, cost derivation, link-state flooding, complexity	`routing.py`	weighted vs hop-count experiment
Cloud Computing	not applicable (no server by design)	–	–

CHAPTER 7

Implementation and Module-Wise Coding

7.1 Development Environment

The system is written in Python 3.10 with `asyncio`, the `cryptography` library and SQLite; the API uses FastAPI, uvicorn and pydantic; the web application uses React 18, Vite and Tailwind; testing uses pytest and pytest-asyncio; version control is Git on GitHub. The source is about 6 500 lines of Python (`mesh/`, `api/`, `sim/`, `story/`), 4 500 lines of tests and 3 100 lines of React.

7.2 Overall Implementation Strategy

The build order followed what blocks what. **Phase 1:** the 52-byte packet and the `Transport` interface, so every track could work against a stable boundary. **Phase 2:** discovery, single-hop relay and the simulated transport, so that routing and reliability could be tested in one process. **Phase 3:** routing, reliability, security and storage, wired into `Node`. **Phase 4:** the API and React application, the scenario runner and metrics, engine fixes found by integration, and the multi-device deployment.

7.3 Module-Wise Implementation

Table 12 gives, for each module, its code reference (file and size), its purpose, input and output, processing logic, dependencies and test evidence.

7.4 Front-End Implementation

Two front ends use the same engine. The **engine web application** (`ui/`, a React PWA) has an overview, a six-beat live demo that drives the real API, a node view with a long-press SOS control, the rescue dashboard (d3-force graph, SOS queue, statistics), and pages for routing, reliability, protocol, security and the simulator; a manifest and service worker make it installable. The **device screens** (`story/node.html`) are one page that becomes a *survivor phone*, a *gateway* view or a *rescue console* by role. The survivor screen is built to be read under stress: a plain status line, need chips, a location field, an SOS button held about 1.2 s to prevent accidental sends, plain-language progress states and nearby phones with signal strength. The rescue console shows SOS cards (sender, text, route, age), counters, a network map and a device list. Screens are shown in Section 10.2.

7.5 Back-End Implementation

The back end is FastAPI with a WebSocket. The engine server (`api/server.py`) exposes the simulation lab (snapshot, reset, kill and revive, link edits, bursts), a node's view, graph and route (weighted and hop-count), `send` and `sos`, the rescue view, statistics, one message's full story, the event log, a protocol inspector, a security playground and attack drill, background scenario jobs, and `/ws`, which streams every logged event. Each device server (`story/node.py`) exposes `/api/me` (everything its screen needs), `/api/sos` and `/api/power`. Request bodies are validated with pydantic, and engine errors map to clear HTTP errors.

7.6 Database Implementation

The three tables of Section 4.6 are created with `CREATE TABLE IF NOT EXISTS`, WAL mode is set at connection time, and the two indexes are created with the schema. Queries used include retained messages by status (`SELECT * FROM messages WHERE status = ?`), a message's timeline (`SELECT * FROM events WHERE msg_id = ? ORDER BY ts_ms`), event counters (`SELECT COUNT(*) FROM events WHERE kind = ?`), and the upsert `INSERT ...ON CONFLICT(msg_id) DO UPDATE`. Every write commits immediately.

Table 12: Modules: code reference, logic, dependencies and tests

Module (lines)	Purpose, input → output, processing, details	Depends on	Tests
`mesh/core/packet.py` (362)	wire format, `Packet` ↔ bytes. `struct` format asserted to be 52 bytes; every field range-checked; `unpack` rejects bad magic and truncation of each region; `remaining_length` frames stream reads; `identity_bytes` is the signed content	–	47 + 4 000 fuzz
transports (461)	bytes between neighbours: simulated (seeded loss, latency), multicast/TCP, and the multi-device `LanTransport` (Section 7.7)	`packet`	15 + 6
`discovery.py` (292)	HELLOs → neighbour table, peer events, link quality over a 20-slot window reported only when it moves by 0.1; `visible_neighbours` range emulation	`transport`	15 + 12
`relay.py` (153)	received packets → deliver, forward or drop (algorithm A4); one drain task over a priority queue	`reliability`	18 + 3
`routing.py` (546)	reports → graph, next hop. `MeshGraph`, `edge_cost`, sequence-numbered gossip with quality, failures, role and key, cached Dijkstra, `RESCUE__` terminals, hop-count switch for the comparison	`discovery`	14 + 14 + 33
`reliability.py` (240)	`DedupSet` (300 s), `MessagePriorityQueue`, `AckTracker` (3 attempts), `StoreForwardQueue` (500 messages, 1800 s)	`packet`	18 + 20
`crypto.py` (131)	AES-256-GCM with nonce prefix and associated data; Ed25519 sign and non-raising verify; one exception type	`cryptography`	19
`store.py` (258)	tables, WAL, upserts, event queries, `events_since`, listing and counting for metrics	`sqlite3`	23
`node.py` (513)	composition root: `send()` encrypts, signs, logs, dispatches; receive verifies, decrypts, logs, ACKs; ACK loop records the failure, recomputes, resends; flush loop expires and retries	all above	27
`sim/`, `api/lab.py`, `api/metrics.py` (3 000)	simulator with a virtual clock and seven scenarios; metrics read from node logs	`node`	5 + 12

7.7 Communication and Concurrency Implementation

The original `UdpTcpTransport` could not carry a complete `Node`: it never learned a HELLO sender's address, and it reported the packet's *original* source instead of the previous hop, which breaks link-state flooding. `lan_transport.py` fixes both. A JSON **topology file** lists every device (role, host, port) and the links (who can hear whom); a device talks only to its configured neighbours, which is the range emulation of the honest statement and satisfies DIS-7, so no multicast is needed. Each connection starts with the previous hop's identity followed by the unmodified packet. A **power switch** makes a device neither send nor hear, like a dead radio. `python -m story.node -id L1` starts one device (engine plus a small web server); five of them, started together on one machine or one per laptop, form the mesh, each with its own SQLite log, port and screen. In the two-laptop demonstration L1–L3 ran on one laptop and L4–L5 on the other, joined through a phone hotspot (Figure 8). Recovery is layered: TCP per hop; ACK, retry and re-route end to end; store-and-forward when retries run out.

7.8 Git Repository Setup

Repository: `https://github.com/tanvisheth1234-wq/miniproject`. **Structure:** `mesh/core/` (engine), `api/`, `sim/`, `story/`, `ui/`, `tests/`, `docs/`, `config/`, with README, CONTRIBUTING and requirements from

the first phase. **Branching strategy:** one branch per phase, each inheriting the previous and never merged back (`develop`, `develop_2`, `develop_3`, `develop_4`, `develop_5`). **Commit conventions:** Conventional Commits (`feat(routing): ...`, `fix(reliability): ...`). **Contribution of members:** Miti Shah built discovery, the relay, the node skeleton and the simulated transport in Phase 2 and, in Phase 3, Dijkstra pathfinding with gateway and rescue handling and route caching, the dedup set and priority queue, ACK tracking with retry and the store-and-forward queue, AES-256-GCM encryption with Ed25519 signing, and the SQLite log; Twissha Shah built the weighted cost function and link-state gossip, the wiring of routing, reliability, crypto and storage into the node, the multi-device program and device screens, the robustness and socket tests, and the evidence scripts; Tanvi Sheth built the packet format, transports, the API, the React application and the scenario runner, and the Phase 4 engine fixes found by integration. Nine of Miti's Phase 3 commits were made from Twissha's laptop and were first recorded under Twissha's Git account; they were re-attributed to Miti on 24 October 2026 with the agreement of all three members, and the correction is recorded in `docs/contribution-log.md`. **Evidence of development history:** Table 13.

Table 13: Commits per team member, read from `git log` over all remote branches (53 commits)

Member	Commits	Share
Miti Shah	12	23 %
Twissha Shah	19	36 %
Tanvi Sheth	22	42 %

Conventional Commits. 47 of 53 commits (89 %) follow the `type(scope): subject` convention of `CONTRIBUTING.md`.

Branches: `main`, `develop`, `develop_2`, `develop_3`, `develop_4`, `develop_5`. **Tags:** `v0.3-phase3`.

7.9 Integration of Modules

`Node` registers one dispatcher on the transport that feeds each packet to discovery (HELLO), gossip (LINK_STATE) and relay (DATA, ACK); each ignores types it does not own. Callbacks connect the pieces: peer-up sets a one-hop route and a graph edge and logs the event; a topology change invalidates cached routes; relay hooks write the message and event log; the ACK loop reports failures to gossip so the cost reacts at once. The API reads the same objects and database, so nothing on screen is computed separately from the engine.

7.10 Implementation Challenges and Resolutions

The significant difficulties (Table 18 gives each cause and retest) were: a single receive callback that let only one of discovery and relay hear packets; topology known only two hops out, with dead nodes still routable; signatures that failed after the first relay (fixed by signing identity fields only); ACKs and retries lost at relays; a hostile packet raising `UnicodeDecodeError`; a hang in `Node.stop()` on Windows; and a transport that could not run a full node (fixed by `LanTransport`).

CHAPTER 8

Scalability, Performance and Resource Management

Every number in this chapter is generated by scripts in `docs/report/evidence/` from stored measurements; none is typed by hand. Two kinds of measurement are never mixed: the *simulator* (real `Node` objects over an in-memory transport with seeded loss and latency on a virtual clock, so the specified 3 s/10 s/4 s timers run in a fraction of a second) and *live devices* (real processes, real TCP, real timers).

8.1 Scalability Requirements

The SRS requires up to fifty nodes, at least 90 % delivery on forty nodes at 20 % loss, convergence within 15 s and recovery within 10 s (Section 1.10); sizes above the five live devices exist only in the simulator.

8.2 Workload and Load Model

Each scenario fixes a topology, a per-link loss rate, 5–40 ms link latency, 10–90 messages (some SOS), a settling period and a failure script, and runs on many seeds, so routing modes are compared on identical traffic. Sweeps vary loss from 0 to 50 % (21 nodes) and size from 10 to 65 nodes (5 % and 20 % loss); throughput uses 100–2 000 simultaneous sends on a three-node line; the live run sent 30 SOS with 8 relay failures and 4 gateway outages.

8.3 Resource Utilization

Simulator: CPU time and Python heap of the whole simulated mesh (Table 15); 65 nodes used about 13 MB. **Live devices:** about 53 MB and 0.4–1.9 % of one core per idle process (Table 16). **Storage:** the SQLite log grows one row per event; retention is bounded at 500 messages. **Network:** transmissions per delivered message are in Table 15.

8.4 Concurrent Processing

A node is single-threaded and cooperative with no locks. On a three-node line with ACKs, 2441 messages per second were delivered at 1 000 simultaneous sends and 1561 per second at 2000 (all ACKed within 2.7 s); the concurrency tests (Chapter 9) confirm exactly-once delivery.

8.5 Scheduling and Synchronization Analysis

Scheduling is by priority, oldest first on ties, re-applied at every hop; its metric is the SOS latency advantage in a deliberately congested scenario (Section 8.7), since on an idle network there is nothing to overtake. No synchronisation primitive is needed because shared state is touched only on the event-loop thread.

8.6 Performance Measurement

One Windows 11 laptop with Python 3.10; live runs used five processes on loopback (HELLO 1 s, neighbour timeout 4 s, link-state 2 s, ACK timeout 3 s, 3 retries). Memory and CPU come from operating-system counters, live latency from polling each device's API every 20 ms (up to 20 ms error), and simulator metrics from queries over the nodes' SQLite logs.

8.7 Performance Results

Reliability under loss (Figure 12a, 21-node grid, 8 seeded runs per point). Delivery stays at or above 98 % to 20 % loss and falls to 61 % at 50 %; each lost ACK costs a 4 s timeout, so median latency goes from tens of milliseconds at 0 % to seconds at 20–30 %.

Table 14: Results of the seven simulator scenarios (weighted routing; delivery ratio shown as mean (min–max) over the runs)

Scenario	Nodes	Loss	Runs	Delivery	Hops	Conv. (s)	Recov. (s)	S&F flushed / queued
baseline_10	10	5%	20	100.0 % (100–100)	1	5.5	–	7/7
collapse_40	40	20%	50	92.6 % (76–100)	4	9.6	8.1	2959/3187
no_gateway	12	5%	20	100.0 % (100–100)	2	5.2	–	201/201
high_loss	20	35%	20	89.1 % (78–100)	2	17.7	–	1007/1048
chain_10hop	11	5%	20	100.0 % (100–100)	9	14.4	–	87/90
hopcount_vs_ weighted	5	45%	20	99.2 % (90–100)	2	5.2	–	–
priority_load	16	0%	20	100.0 % (100–100)	2	7.6	–	1356/1356

Hops = median hop count of delivered messages. Conv. = mean seconds until every node's graph contains every node. Recov. = mean seconds from relay death until the sender has a route avoiding it. S&F counts are event counts summed over all runs: a message re-queued after a failed retry is counted each time, and messages still held when a run ends are not flushed.

Scenarios. The 40-node collapse scenario (20 % loss, a relay on the active path killed at $t = 10$ s, the gateway killed at 25 s and revived at 60 s) delivered 92.6 % on average over 50 seeded runs (worst 76 %, best 100 %). At 35 % loss (outside the specified condition) delivery was 89.1 %. **Weighted versus hop-count**

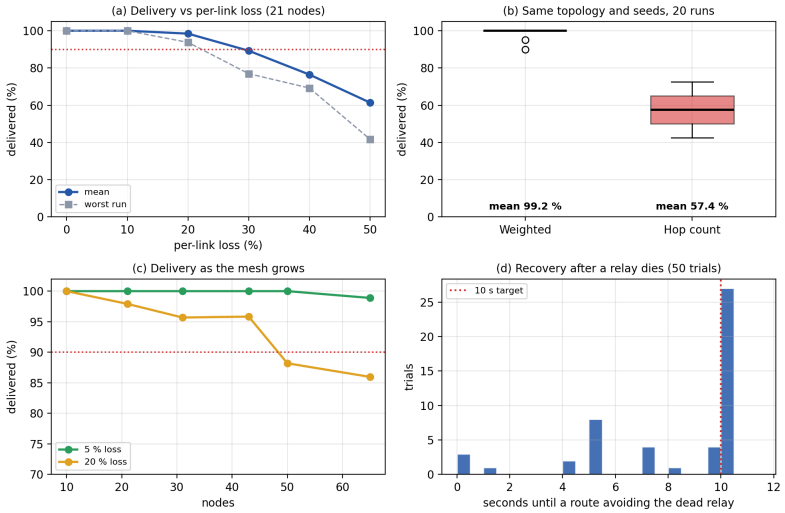


Figure 12: (a) Delivery against per-link loss. (b) Weighted versus hop-count routing on identical seeds. (c) Delivery as the mesh grows. (d) Recovery time after a relay death, 50 trials.

routing (Figure 12b). In the designed topology the 2-hop shortcut crosses a link with 45 % loss and the 3-hop path is clean; messages are best-effort, so a poor route shows up directly as loss. Weighted routing delivered 99.2 % and hop-count routing 57.4 %, and weighted won on 20 of 20 identical seeds. It is a *designed* topology that exposes hop count's weakness, so it shows that the cost function avoids a lossy shortcut, not that the same gap occurs on every mesh.

Priority under load. The congested scenario (16 nodes, 90 messages to the same destination, 15 % SOS) gives an SOS a median latency 4.4× lower than routine traffic. Only here do SOS and routine messages take the same trip, so only here is the comparison fair.

Scalability. At 5 % loss delivery is 100 % to 50 nodes and 98.9 % at 65; at 20 % loss it holds above 95 % to about 43 nodes and falls to 88.2 % at 50 and 86.0 % at 65. Transmissions per delivered message grow roughly linearly with size, not quadratically as flooding would.

Recovery and convergence are quantised by design constants (Figure 12d): convergence comes in mul-

Table 15: Scalability, 10 to 65 simulated nodes (6 runs per size and loss rate)

Nodes	Links	Delivery at 5% loss	Delivery at 20% loss	Tx per delivered (5%)	Tx per delivered (20%)	Conv. max at 20% (s)	CPU (s, 5%)	Heap (MB, 5%)
10	13	100.0 %	100.0 %	5.2	7.8	10.2	3	1.1
21	32	100.0 %	97.9 %	7.7	15.7	15.2	12	2.3
31	50	100.0 %	95.7 %	10.2	23.4	10.2	29	4.0
43	72	100.0 %	95.8 %	12.7	24.6	15.2	52	6.9
50	85	100.0 %	88.2 %	14.4	33.3	10.2	69	8.5
65	113	98.9 %	86.0 %	20.8	39.3	15.2	126	12.9

Nodes and links include the gateway and rescue post added to the grid. Tx = hop-by-hop data transmissions. CPU and heap are for the whole simulated mesh in one process on a virtual clock, not the cost of one real device.

tiples of the 5 s gossip interval, and recovery takes under 10 s in 23 of 50 trials (ACK failures penalise the link at once) and 10.2–10.25 s in the other 27, bounded by the 10 s heartbeat timeout.

Live devices. Table 16 gives the measurements on five real processes over real TCP sockets.

Table 16: Live devices: 5 operating-system processes, real TCP sockets, real timers, on one laptop (loopback)

Measurement	Result
Process start → every device knows every device	3.8 s
SOS arrives at the rescue device (30/30 delivered)	median 47 ms, 95th pct 102 ms, max 281 ms
ACK back at the sender	median 63 ms, 95th pct 149 ms
Relay switched off → sender re-routes (8 trials)	median 4.0 s, max 4.5 s; every later SOS delivered
Gateway down, 5 SOS held; gateway back (4 trials)	100% flushed, median 4.5 s, max 6.1 s
Device API response (200 requests)	median 16 ms, 95th pct 31 ms
Per-process memory and CPU, idle	53–54 MB; 0.4–1.9% of one core
Per-process CPU while sending SOS (includes the harness polling every 20 ms)	up to 19% of one core

Timer settings: HELLO 1 s, neighbour timeout 4 s, link-state 2 s, ACK timeout 3 s. Latency is measured by polling each device's own API every 20 ms (up to 20 ms of error). Wireless delay and loss are not included.

8.8 Bottleneck Identification

Failure detection (recovery is bounded by the 10 s heartbeat timeout), the 4 s ACK timeout under loss, the 5 s link-state interval, one SQLite commit per event and wholesale route-cache invalidation.

8.9 Optimization and Improvement

No optimisation was applied after these measurements, so there is no before-and-after comparison. The data point to a neighbour timeout near 6–7 s (recovery inside 10 s), an immediate link-state report on topology change, an adaptive ACK timeout and per-destination cache invalidation.

8.10 Cloud Deployment and Resource Utilization

Not applicable: the system has no server and was not deployed on cloud infrastructure.

8.11 Scalability Limitations

Tested up to 65 simulated nodes (98.9 % delivery at 5 % loss, 86.0 % at 20 %) and 5 live devices. More nodes, wireless interference and mobility were not tested; capacity beyond these sizes is a projection.

CHAPTER 9
Testing and Validation

9.1 Testing Strategy

Testing is layered (unit, integration, functional, non-functional, boundary, concurrency and live devices); the simulated transport makes routing and reliability tests deterministic, so a failure reproduces from a seed. The 312 test cases were run completely 3 times. Complete suite runs: 3; fully green: 2. Inconsistent test(s): `test_api::test_node_view_reports_peers_with_hop_distance`.

9.2 Unit Testing

Each module is tested alone: the packet format (47 tests: round trips, field ranges, truncation of each region, bad magic, signable bytes), the cost function and graph, Dijkstra with caching and `RESCUE__`, gossip, discovery and link quality, the dedup set, priority queue, ACK tracker and store-and-forward bounds, AES-GCM and Ed25519, the SQLite log, the relay and the simulated transport.

9.3 Integration Testing

`test_node` (27 tests) covers discovery and relay on one transport, multi-hop origination, ACK, the message lifecycle, end-to-end encryption and signing, retry exhaustion, eviction and expiry; `test_ack_multihop` checks that ACKs and retries cross relays correctly; `test_api` drives front end, back end and engine through FastAPI and its WebSocket; `test_lan_transport` runs a three-node mesh over real TCP. The screens were checked by running them (Section 10.2).

9.4 Functional Testing

Table 4 summarises how the requirements are verified. 68 functional requirements: 52 verified by automated tests alone, 10 by automated tests plus a manual check of the screen, 2 partially implemented, 2 by code inspection, 2 by manual check only. All mapped tests passed in every complete run except one timing-sensitive test that failed once (Section 9.9). SNF-3 (immediate attempt on a new neighbour) and LOG-5 (history filter on screen) are only partly met.

9.5 Non-Functional Testing

Reliability: delivery at 0–50 % loss (Figure 12a). **Scalability:** 10 to 65 nodes (Table 15). **Performance:** latency by priority (Section 8.7), throughput and live latency (Table 16). **Availability:** recovery over 50 simulated trials and 8 live failovers. **Security:** a relay without the key never sees plaintext; tampered ciphertext, a wrong key, a forged signature and an unknown signer are rejected; signatures verify across relays. **Usability:** one 1.2 s hold on one button with plain-language status; no timed study with participants (Section 9.11).

9.6 Boundary and Invalid-Input Testing

Tests cover empty, header-only and truncated input; a bad magic value; out-of-range TTL and lengths; over-long paths and node ids; wrong-length signatures and key sizes; ciphertext shorter than the nonce; and a send to an unknown destination (held, not an exception). `test_robustness` feeds 3 000 random and 1 000 bit-flipped packets to the parser and 200 garbage buffers to a running node, which must only reject them with `PacketError` and keep working.

9.7 Concurrency and Stress Testing

200 simultaneous sends over two hops arrived exactly once with every ACK returned; 50 SOS and 50 routine messages from two senders all arrived intact; a packet replayed five times was delivered once. Throughput to

2 000 sends and the congested priority scenario complete the stress tests; with all shared state on one thread, races and deadlocks cannot occur.

9.8 Test Case Specification

Table 17 specifies eight representative test cases with their requirement, preconditions, expected and actual results and evidence.

Table 17: Representative test cases

ID	Req.	Scenario, input and preconditions	Expected result	Actual result and evidence	Result
TC-01	RTE-3, 4	short path over a lossy link, long path over clean links	Dijkstra takes the clean path	as expected; `test_dijkstra`	Pass
TC-02	PRI-3	SOS reaches a relay with routine packets queued	SOS forwarded first	as expected; `test_relay`	Pass
TC-03	REL-5, 6	ACK lost; sender times out	route recomputed, resent, destination sees it once	as expected; `test_ack_multihop`	Pass
TC-04	SNF-1, 2	no route; a route later appears	held, then flushed and delivered	as expected; live: 4 of 4 outages flushed (Table 16)	Pass
TC-05	FWD-6	3 000 random byte strings	only `PacketError`; node survives	one defect found and fixed (D4); `test_robustness`	Pass
TC-06	K1	40 nodes, 20 % loss, relay and gateway failures, 50 seeds	delivery $\geq$ 90 %	92.6 % (worst run 76 %); Table 14	Pass
TC-07	K3	kill the relay on the active path, 50 trials	re-route $\leq$ 10 s	mean 8.1 s; 23 of 50 within 10 s; worst 10.25 s; Figure 12	Partial
TC-08	K4, K6	congested radio; weighted versus hop count on identical seeds	SOS $\geq$ 3$\times$ faster; weighted delivers more	4.4$\times$; 99.2 % vs 57.4 %; Section 8.7	Pass

9.9 Defect Identification and Resolution

Table 18 records each significant defect, its cause, fix and retest; D4 and D5 were fixed before the Chapter 8 measurements.

9.10 Validation Against KPIs

Table 19 compares each target of Table 1 with the measured result. K4–K6 and K10 are met with margin and K1 in the specified condition; K2 and K3 hold on average but miss by a fraction of a second in some runs (timer constants); K7 holds at 5 % loss, not at 20 % above about 43 nodes; K9 has one flaky test.

9.11 Limitations and Untested Scenarios

No radio hardware: interference, fading and mobility were not tested. **Live timing is loopback;** the two-laptop run was not timed. **No usability study** and **no automated UI tests**. **Scale:** nothing above 65 nodes, no soak test. **Relays do not retain. Security:** one pre-shared key serves the mesh; no external review.

Table 18: Significant defects, causes, corrective actions and retesting

ID	Found by	Defect and cause	Fix and retest
D1	integration test	one receive callback: only the last of discovery and relay heard packets	single dispatcher; regression test
D2	Phase 4 integration	dedup keyed on message id dropped every ACK at the first relay and swallowed retries	key on type, id and attempt; retry counter in the reserved byte; new tests
D3	Phase 4 integration	dead nodes stayed routable: newer reports merged with old ones	a newer report replaces the origin's edges; tests
D4	fuzz test	a non-ASCII node id made `Packet.unpack` raise `UnicodeDecodeError`, which receivers do not catch	decode failures raise `PacketError`; fuzz tests pass
D5	repeated socket-test runs	`Node.stop()` hung in about a quarter of runs on Windows: `wait_for` swallowed a cancellation	helper on `asyncio.wait`; 15/15 and 40/40 runs clean
D6	repeated full-suite runs	two timing-sensitive tests (`test_api`, `test_ack_multihop`) failed in 2 and 1 of 6 runs under load; both pass alone	open: replace fixed sleeps with condition polling

Table 19: Validation of the key performance indicators of Table 1 against measured data

KPI	Target	Measured	Verdict
K1 Delivery	$\geq 90\,\%$ (40 nodes, 20 % loss, failures injected)	92.6 % mean over 50 runs (worst run 76 %). Without failures, a 43-node grid at 20 % loss: 95.8 %.	Met
K2 Convergence	$\leq 15\,\mathrm{s}$	collapse_40: mean 9.6 s; 47 of 50 runs $\leq 15\,\mathrm{s}$; worst 15.25 s. Mean 17.7 s in the 35 % loss scenario. Values come in multiples of the 5 s gossip interval. Live devices (2 s gossip): 3.8 s from process start.	Met in 47/50 runs; not at 35 % loss
K3 Recovery	$\leq 10\,\mathrm{s}$	mean 8.1 s, median 10.2 s; 23 of 50 trials $\leq 10\,\mathrm{s}$; the other 27 took 10.2–10.25 s, i.e. the 10 s heartbeat timeout plus the 0.25 s sampling step. Live devices (4 s timeout): median 4.0 s, max 4.5 s over 8 trials, every message delivered	Mean met; 27/50 trials miss by $\leq$ 0.25 s
K4 SOS advantage	$\geq 3\times$	4.4× (median routine / median SOS)	Met
K5 Store-and-forward	100 % flushed	201/201 in the no-gateway scenario; 100 % of live trials	Met
K6 Weighted vs hop count	Weighted better	99.2 % vs 57.4 %	Met
K7 Scale	≥ 50 nodes within the targets	50 nodes at 5 % loss: 100.0 % delivery; 50 nodes at 20 % loss: 88.2 %; largest tested 65 nodes: 98.9 % (5 %) and 86.0 % (20 %)	Met at 5 % loss; not at 20 %
K8 UI freshness	$\leq 2\,\mathrm{s}$	API response median 16 ms; screens poll every 0.7 s	Met
K9 Correctness	100 % passed	311/312 passed in all 3 runs; 1 timing-sensitive test failed once	Met except one flaky test
K10 Security	All hold	relay cannot decrypt, forged signature rejected, tampered payload rejected: automated tests pass	Met

CHAPTER 10

Results, Demonstration and Discussion

10.1 Implemented Features

Table 20 lists what was implemented, what is partial and what was not built, with the evidence for each.

Table 20: Feature status

Feature	Status	Evidence
Discovery, link-state gossip, weighted Dijkstra, priority, TTL and dedup, ACK and retry, store-and-forward, AES-GCM, Ed25519, SQLite log	Implemented	312 tests; Chapters 8, 9
Simulator, metrics, API, engine web application, survivor and rescue screens	Implemented	7 scenarios to 65 nodes; 12 API tests; Section 10.2
Multi-device deployment over real sockets	Implemented	5 processes measured; two-laptop demonstration
Immediate flush on a new neighbour; history filter	Partial	SNF-3, LOG-5
BLE or Wi-Fi Direct radio; per-pair key agreement; timed usability study	Not implemented	out of scope or future work

10.2 Screenshots and Demonstration Evidence

All screenshots are from the running system. Figures 13 and 14 come from the five-process live deployment in the demonstrated story: an SOS is delivered over a relay; the gateway fails and the next SOS is held; the gateway returns and the held message is delivered. The mesh was also run on two physical laptops joined by a phone hotspot (Figure 8): an SOS was delivered across both machines, the route changed when a relay was switched off, and a message held while the gateway was off was delivered when it returned. This run was a functional demonstration and was not timed.

10.3 Functional Results

Of the 68 requirements, 52 are verified by automated tests alone, 10 with a manual screen check, 2 by inspection, 2 manually, and 2 are partial (Table 4). The three key stories ran on real devices: an SOS reaches rescue over several hops, the mesh re-routes around a failed relay, and a message with no route is held and later delivered.

10.4 Performance Results

Table 19 collects the measured KPIs: 92.6 % delivery on 40 nodes at 20 % loss with failures, an SOS 4.4× faster under load, live SOS delivery in a median 47 ms, relay failover in about 4 s and held messages flushed in about 4.5 s.

10.5 Comparison with Existing Systems

Other systems were not run, so the comparison uses documented properties (Table 3) and the one fair measurable baseline, hop-count routing on identical seeds (99.2 % against 57.4 %). No claim is made about the delivery of Bridgefy, Briar, Meshtastic or Serval, which publish no comparable measurements.

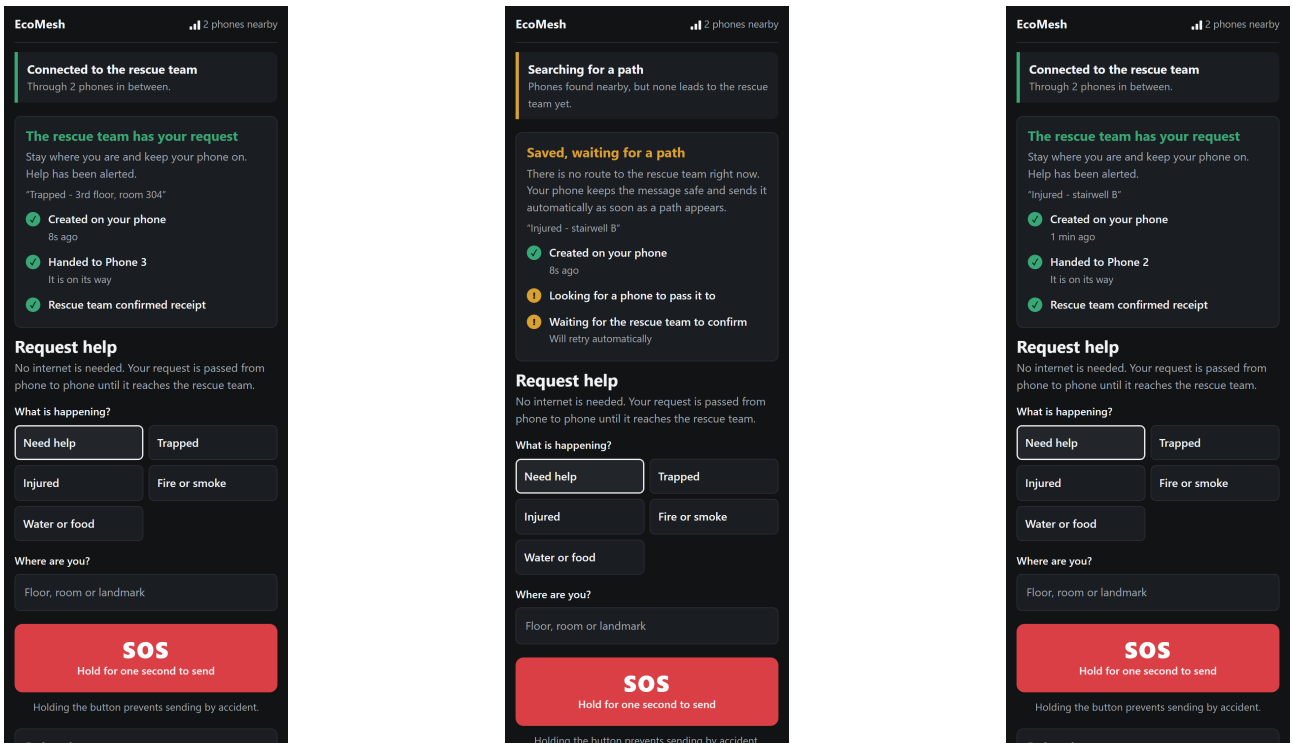


Figure 13: The survivor's screen. Left: SOS delivered over a relay phone and the Gateway. Centre: the gateway has failed; the screen says "Searching for a path" and the message is "Saved, waiting for a path". Right: the gateway is back and the held message is delivered.

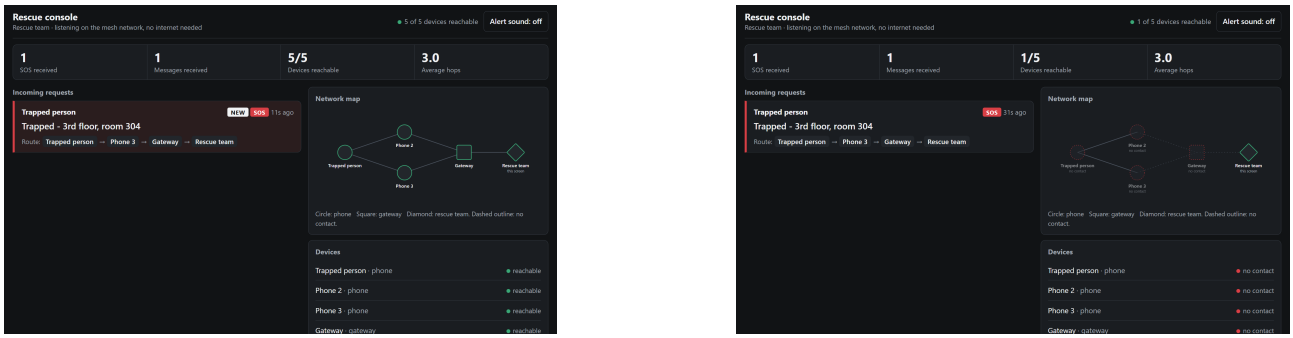


Figure 14: The rescue console. Left: SOS cards with sender, text, age and route, counters, network map and device list. Right: with the gateway down the rescue device hears nobody ("1 of 5 devices reachable"); earlier messages remain visible.

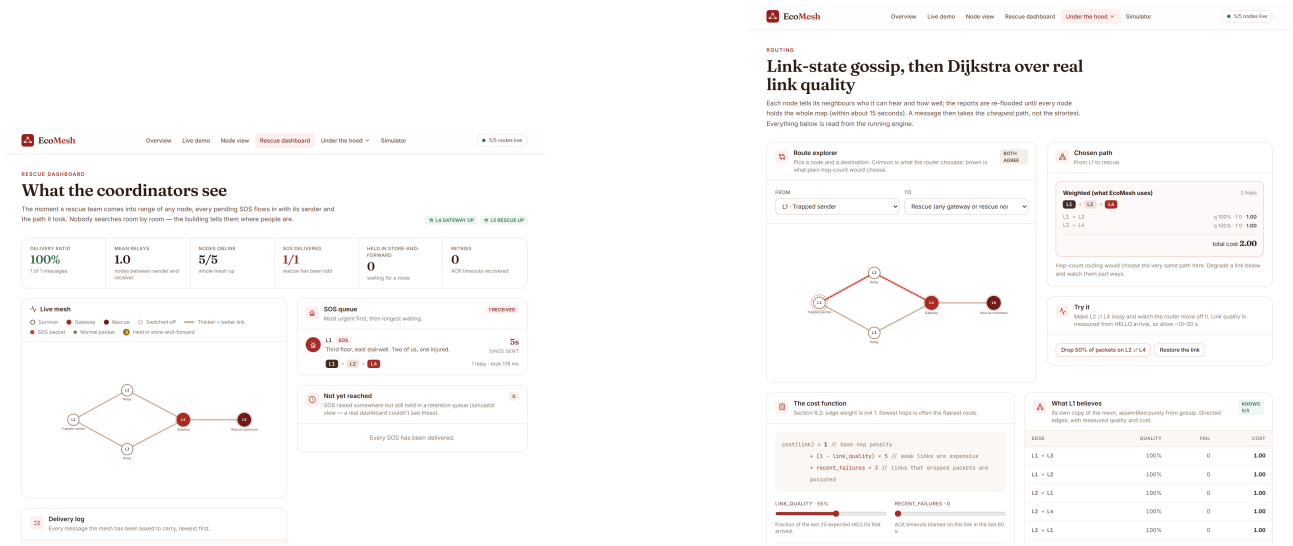


Figure 15: The engine web application. Left: rescue dashboard (delivery ratio, nodes online, live mesh, SOS queue). Right: routing page (weighted versus hop-count route, cost function, a node's view of the graph). A six-beat live demonstration page drives the same API.

10.6 Achievement of Objectives

Table 21 maps each objective of Section 1.4 to its evidence and status.

Table 21: Objectives, supporting evidence and status

Obj.	Evidence	Status
O1	live convergence 3.8 s; simulator 5–15 s in multiples of the gossip interval	Achieved (47 of 50 runs $\leq$ 15 s)
O2	99.2 % vs 57.4 % on 20 identical seeds (Figure 12b)	Achieved
O3	4.4× lower median latency under load (Section 8.7)	Achieved
O4	92.6 % at 40 nodes and 20 % loss; recovery mean 8.1 s, 23 of 50 trials within 10 s	Partly achieved
O5	201 of 201 flushed in simulation; 4 of 4 live outages	Achieved
O6	security tests (Section 9.5)	Achieved with a pre-shared key
O7	every metric is a query over per-node logs	Achieved
O8	web application, device screens, API tests, five processes, two laptops	Achieved

10.7 Discussion of Findings

Hop count is a poor proxy for delivery: a weak shortcut loses almost half the traffic under hop count and almost none under the weighted cost. Timer constants, not algorithms, set recovery and convergence (they cluster at the heartbeat timeout and the gossip interval), which shows exactly what to tune. Loss is paid for in tail latency, and priority matters only once a link queues, which is the disaster condition. Testing found real defects in code we believed finished (Section 9.9).

10.8 Practical Applicability

A deployment needs devices running the node program, a topology file (or multicast discovery), a gateway or rescue device and a shared network segment. It is usable today as a campus, hospital or building safety system on laptops; a Bluetooth LE or Wi-Fi Direct transport would make it phone-to-phone.

CHAPTER 11
Conclusion and Future Scope

11.1 Conclusion

Every mainstream messaging tool fails when its infrastructure fails, although devices in a stricken building can still reach each other. We built EcoMesh, a peer-to-peer mesh in which every device is an endpoint and a relay, with a complete engine, a simulator that measures it, a REST/WebSocket interface, two front ends and a multi-device deployment over TCP. Weighted routing delivered 99.2 % of messages where hop-count routing delivered 57.4 % on identical seeds; an SOS had a 4.4× lower median latency under load; a 40-node mesh at 20 % loss with injected failures delivered 92.6 % over 50 runs; and on five live processes an SOS arrived in a median of 47 ms, the sender re-routed around a failed relay in about 4 s, and every message held during a gateway outage was delivered when the gateway returned.

11.2 Achievement of Objectives

Objectives O1–O3 and O5–O8 were achieved. O4 was achieved for delivery but only partly for recovery time, where 27 of 50 trials exceeded 10 s by at most 0.25 s, the heartbeat timeout (Table 21).

11.3 Major Engineering Contributions

A transport-agnostic engine that runs unchanged on a simulator and on real sockets; a derived link-quality cost function with an immediate failure penalty and network-wide re-flooding; per-hop priority with a bounded queue that never evicts an SOS for routine traffic; signatures that survive multi-hop relaying; and a measurement framework in which every number in this report is generated from per-node logs and stored results.

11.4 Current Limitations

As detailed in Section 9.11: a neighbour-restricted LAN stands in for a radio; one pre-shared key serves the mesh; relays do not retain; timer constants quantise recovery and convergence; SNF-3 and LOG-5 are partly met; there is no usability study or automated UI test; and scale was tested to 65 simulated nodes.

11.5 Future Enhancements

(1) A Bluetooth LE or Wi-Fi Direct transport and an Android client speaking the same protocol. (2) Shorter and adaptive timers: a neighbour timeout near 6–7 s, an immediate report on topology change and an ACK timeout derived from the measured round-trip time. (3) X25519 per-pair key agreement. (4) Short retention at relays and an immediate flush on a new neighbour. (5) Per-destination route invalidation. (6) A usability study, automated browser tests, condition polling in timing-sensitive tests, and tests beyond 65 nodes.

11.6 Lessons Learned

Build the simulator first and draw the transport boundary before the layers: it paid for itself in testing and in deployment. Integration finds what unit tests do not (the multi-hop ACK bug survived every single-node test). Look at distributions as well as means: recovery clustered at the heartbeat timeout, which the mean hid. Record who built what from the first week, with consistent Git identities.

References

[1] M. R. Albrecht, J. Blasco, R. B. Jensen and L. Mareňková, "Mesh Messaging in Large-scale Protests: Breaking Bridgefy," in *Topics in Cryptology – CT-RSA 2021*, Lecture Notes in Computer Science, vol. 12704, Springer, 2021, pp. 375–398.

[2] C. E. Perkins and P. Bhagwat, "Highly dynamic Destination-Sequenced Distance-Vector routing (DSDV) for mobile computers," in *Proc. ACM SIGCOMM*, 1994, pp. 234–244.

[3] D. Johnson, Y. Hu and D. Maltz, "The Dynamic Source Routing Protocol (DSR) for Mobile Ad Hoc Networks for IPv4," RFC 4728, IETF, Feb. 2007.

[4] C. Perkins, E. Belding-Royer and S. Das, "Ad hoc On-Demand Distance Vector (AODV) Routing," RFC 3561, IETF, July 2003.

[5] T. Clausen and P. Jacquet (eds.), "Optimized Link State Routing Protocol (OLSR)," RFC 3626, IETF, Oct. 2003.

[6] J. Moy, "OSPF Version 2," RFC 2328, IETF, Apr. 1998.

[7] J. Chroboczek and D. Schinazi, "The Babel Routing Protocol," RFC 8966, IETF, Jan. 2021.

[8] D. S. J. De Couto, D. Aguayo, J. Bicket and R. Morris, "A high-throughput path metric for multi-hop wireless routing," in *Proc. ACM MobiCom*, 2003, pp. 134–146.

[9] A. Vahdat and D. Becker, "Epidemic Routing for Partially-Connected Ad Hoc Networks," Technical Report CS-2000-06, Duke University, 2000.

[10] K. Fall, "A delay-tolerant network architecture for challenged internets," in *Proc. ACM SIGCOMM*, 2003, pp. 27–34.

[11] V. Cerf *et al.*, "Delay-Tolerant Networking Architecture," RFC 4838, IETF, Apr. 2007.

[12] D. G. Reina, M. Askalani, S. L. Toral, F. Barrero, E. Asimakopoulou and N. Bessis, "A survey on multihop ad hoc networks for disaster response scenarios," *International Journal of Distributed Sensor Networks*, vol. 11, no. 10, art. 647037, 2015.

[13] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[14] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe and B.-Y. Yang, "High-speed high-security signatures," *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.

[15] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[16] The Briar Project, "Briar: secure messaging anywhere – how it works," `https://briarproject.org`, accessed 2026.

[17] Meshtastic, "Mesh Broadcast Algorithm" and "Why Meshtastic uses managed flood routing," documentation, `https://meshtastic.org/docs/overview/mesh-algo/`, accessed 2026.

[18] The Serval Project, "Serval: communications for disaster and remote areas" (including the Rhizome data-distribution protocol), project documentation, `https://www.servalproject.org`, accessed 2026.

Appendix: Research Paper / Poster and Plagiarism Report

As the department template requires, the end-semester research paper or poster (with, if accepted for publication, the acceptance receipt or certificate) and the plagiarism report of this report, generated with the college-approved checker, are inserted after this page.

[Insert the research paper or poster here]

[Insert the plagiarism report here]